

One Nation, One Material Code

How the system works, end to end — what every stage takes in, what it does, what it hands to the next stage, and who on the team owns it. Read this before writing any code.

MINISTRY

Petroleum & Natural Gas

VERSION

30 Aug 2026

DEPARTMENT

Chennai Petroleum (CPCL)

TEAM

Overwatch · VCET

01 The problem, in plain terms

Why a national material code does not exist yet

Every large public company — ONGC, IOCL, CPCL, BPCL, SAIL, NTPC — keeps a catalogue of everything it buys. That catalogue is called the **material master**. Each item in it has a code, a description, a unit, and a category.

The same physical item is written differently in every company, and often written more than once inside a single company. Here is one ball bearing, in four catalogues:

CPCL	100004521	BRG BALL DP GRV 6205 2RS SKF
IOCL	MAT-88213	BEARING,BALL,6205-2RS,SKF
ONGC	5000123	BALL BRG SKF 6205 2RS DEEP GROOVE
BPCL	BRG-4471	DEEP GROOVE BALL BEARING 6205 2RS

All four are the same bearing. To a computer they are four different items.

Why the descriptions look like that

This is not carelessness. Older SAP systems gave the description field (`MAKT-MAKTX`) only **40 characters**. A storekeeper who needed to write *"Ball Bearing, Deep Groove, 25mm bore, 52mm OD, 15mm width, rubber sealed, SKF"* had to cut it down to fit. Everybody cut it down differently. The abbreviations are a side effect of a field length from thirty years ago.

What it costs

WHAT GOES WRONG	WHAT IT ACTUALLY COSTS
Same item under six codes	Six separate safety stocks. One warehouse holds 400 spare while another reorders as "out of stock"
Purchases split across codes	Six small orders at list price instead of one large order with a volume discount
Spend analytics is wrong	"How much did we spend on bearings last year?" has no answer
No sharing between companies	ONGC has 400 sitting idle while IOCL raises an emergency purchase. Neither knows.
Dead stock	Old codes nobody remembers. Material sits until it is scrapped.

Published case studies of material master standardisation report roughly 5–7% **inventory cost reduction**. Across all CPSEs that is thousands of crores.

TWO SEPARATE BLINDNESSES — KEEP THEM APART

Inside one company: ONGC has full access to its own data, but cannot tell that six of its own codes are the same item. This is a *technical* problem.

Between companies: ONGC cannot see what IOCL paid, because no company will publish its prices and vendor list. This is a *trust* problem, and it is the real reason a national material master has never been built.

02

The core idea

Match on specifications, not on spelling

Almost every team attacking this will compare descriptions and ask "*how similar is this text?*" That fails in both directions, and the two failures are the whole reason this project exists.

DIFFERENT WORDS, SAME ITEM

```
"SKF 6205-2RS Deep Groove Ball Bearing"  
"FAG 6205-2RSR bearing, 25x52x15mm"
```

```
text similarity ..... 0.31  
a fuzzy matcher ..... MISSES IT
```

6205 is an ISO designation. Any manufacturer's 6205 is 25 mm bore, 52 mm outer diameter, 15 mm wide. These two bearings are physically interchangeable.

SAME WORDS, DIFFERENT ITEM

```
"GASKET SPIRAL WOUND SS316 4IN 150#"  
"GASKET SPIRAL WOUND SS304 4IN 150#"
```

```
text similarity ..... 0.97  
a fuzzy matcher ..... MERGES THEM  
(wrong)
```

SS316 contains molybdenum and resists corrosion; SS304 does not. In a corrosive refinery line the wrong one leaks. These must never be merged.

So text similarity is not the answer — it is not even the right question. Instead we pull structured specifications out of each description first, and compare those:

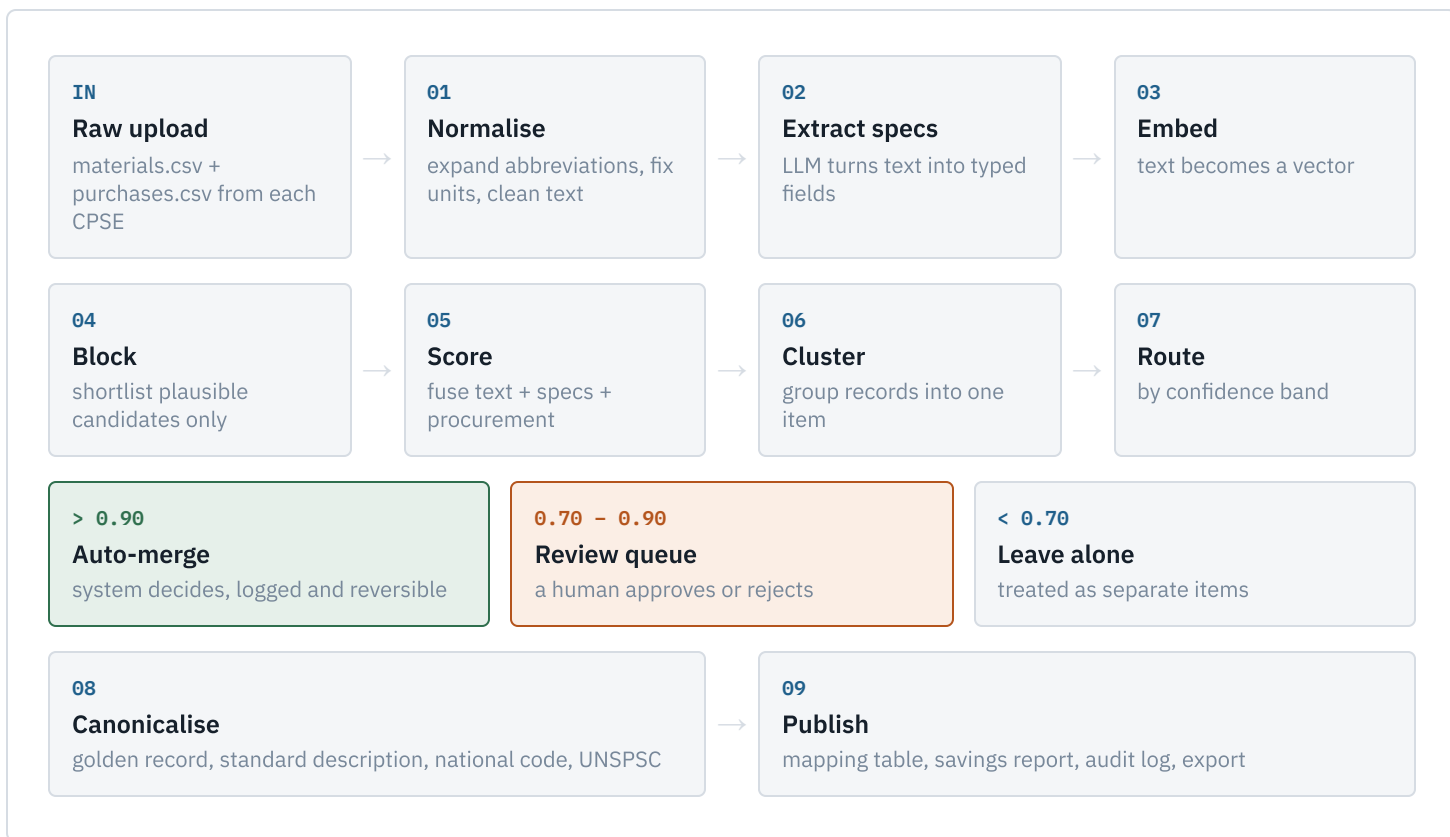
	RECORD A		RECORD B	
category	:	bearing	✓	bearing
sub_type	:	deep_groove	✓	deep_groove
iso_designation	:	6205	✓	6205
bore_mm	:	25	✓	25
od_mm	:	52	✓	52
width_mm	:	15	✓	15
seal_type	:	2RS	✓	2RS
brand	:	SKF	⊗	FAG <- deliberately ignored
score 0.97 -> MATCH				

We do not ask whether two descriptions look alike.
We ask whether two items are the same thing.

Brand is ignored on purpose, because functional equivalence does not care who made it. And a mismatch on a critical field like material grade or pressure rating kills the match outright, no matter how well everything else lines up.

03 The whole flow, at a glance

Nine stages, then three exits



Read it as a conveyor belt. Each stage only needs the output of the one before it, which is why the work splits cleanly across people, and why a half-finished pipeline still runs end to end on a small dataset.

04 Stage by stage

What goes in, what happens, what comes out

IN

Two files per CPSE, exported from their SAP system. Nobody types these by hand — it is a report a store manager runs.

```
materials.csv  code, description, uom, material_group, plant
purchases.csv code, vendor, unit_price, qty, po_date, plant
```

DOES

Reads the files, checks required columns exist, tags every row with which CPSE it came from, and stores the raw text exactly as received. The raw copy is never modified after this point.

OUT

One `materials` table holding every record from every CPSE, plus a `purchases` table linked to it by source code.

IN Raw description text and unit of measurement.

DOES Three passes, all rule-based, no AI:

Expand abbreviations using a dictionary — BRG → BEARING , M.S. → MILD STEEL , DP GRV → DEEP GROOVE , NB → NOMINAL BORE .

Canonicalise units. The problem statement calls this out specifically, so it is a named feature, not a side effect:

EA, NOS, NO, PCS, PC	->	EACH
M, MTR, MTRS, MTS	->	METRE
KG, KGS, K.G.	->	KILOGRAM
L, LTR, LIT	->	LITRE

Watch for pack sizes — if one CPSE prices per piece and another per box of 100, comparing prices directly is wrong.

Clean text — casing, stray punctuation, repeated spaces, Ø → DIA .

OUT A clean_description and a canonical_uom beside every raw record. Both stored; nothing overwritten.

IN One cleaned description.

DOES Sends the description to an LLM with a strict schema for its category, and gets back typed attributes with a confidence score on every field.

```
INPUT    "BRG BALL DP GRV 6205 2RS SKF"

OUTPUT  {
  "category": "bearing",
  "sub_type": "deep_groove_ball",
  "iso_designation": "6205",
  "bore_mm": 25,
  "od_mm": 52,
  "width_mm": 15,
  "seal_type": "2RS",
  "brand": "SKF",
  "confidence": {
    "iso_designation": 0.98,
    "bore_mm": 0.85,      <- derived, not stated
    "brand": 0.99
  }
}
```

How does the model know 6205 means 25 mm? Because bearing numbering is a published ISO standard and the model has read thousands of engineering catalogues. It is genuine public knowledge, not a guess. A regular expression could never derive it.

Three things this stage must do:

- **A separate schema per category.** A pipe schema and a valve schema have almost nothing in common. One generic schema will not work.
- **Confidence per field.** A value read straight out of the text scores high; a value derived from a standard scores lower; a guess scores low and carries less weight later.
- **Say "unknown" rather than guess.** A made-up spec that then drives a merge is the worst failure this system can produce. Unknown goes to review.

A regex-based fallback runs when the LLM is unavailable. It catches patterns like `SCH\s?(\d+)` and `SS\s?(304|316)` and handles roughly 60% of records, which is enough to keep the system running.

OUT

A JSON specification object per record, saved permanently in the database.

THE COST RULE THAT MUST NEVER BREAK

The LLM runs **once per record, at ingestion**. It is never called to compare a pair of records.

Once per record	1,000,000 calls	~3 hrs batched	run once, result stored
Once per pair	20,000,000 calls	~55 hrs	run every single time

Per-pair LLM calls make the system impossible at real scale. If any design starts drifting that way, stop and raise it.

03

Embeddings

RASHMIT

IN

Cleaned description text.

DOES

Converts each description into a list of numbers — a vector — such that texts with similar meaning end up close together in that number space.

"ball bearing"	->	[0.21, -0.05, 0.88, ...]	384 numbers
"roller bearing"	->	[0.19, -0.03, 0.85, ...]	close
"stainless steel pipe"	->	[-0.44, 0.71, 0.02, ...]	far

Closeness is measured with **cosine similarity** — the angle between two vectors. Identical direction is 1.0, unrelated is 0. It is only multiplication and addition, so a GPU does millions of these per second. That is exactly why this scales and an LLM does not.

Run locally with `sentence-transformers` on Rashmit's GPU.

OUT

One vector per record, stored in the database next to the record.

IN All records, with their categories and vectors.

DOES Decides which pairs are even worth scoring. Comparing everything against everything is not an option:

```
1,000,000 records -> 500,000,000,000 pairs ~6 days of compute
```

A bearing never needs to be compared against a gasket. So we narrow first:

```
all records
-> group by category          bearings: 12,000
-> group by key attribute     bearings, 25mm bore: 300
-> plus top-50 nearest vectors

result: ~20,000,000 pairs instead of 500,000,000,000
```

Two numbers must be measured here and put on the pitch slide:

- **Reduction ratio** — how much of the work we eliminated. Target above 99.9%.
- **Pair completeness** — how many genuine matches survived the narrowing. Target above 98%.

Blocking too tightly loses real matches. Blocking too loosely wastes time without improving accuracy. Both numbers must be tracked together.

OUT A list of candidate pairs.

IN Candidate pairs, each with both records' specs, vectors and purchase history.

DOES Produces one number between 0 and 1 for each pair, by combining three independent kinds of evidence (detailed in section 05).

```
score = w1 · text_similarity
       + w2 · spec_agreement      <- heaviest
       + w3 · procurement_similarity

any hard blocker -> score = 0
```

Three rules matter more than the weights:

- **Hard blockers.** Material grade, pressure rating and key dimensions are veto fields. A mismatch there is an instant zero however well the rest matches.
- **Ignored fields.** Brand and vendor part number never count toward a match — that is what lets SKF and FAG resolve to one item.
- **Missing is not mismatched.** If one record states a thickness and the other is silent, that is absence of information, not evidence of difference. No penalty.

This whole stage is arithmetic. No AI runs here, which is what makes every score reproducible and explainable.

OUT A score per pair, stored with the individual signal values that produced it, so the review screen can show *why*.

IN Scored pairs above the threshold.

DOES Turns pairs into groups. Every record is a node, every strong pair is an edge, and each connected group becomes one item. Implemented with union-find.

```
A --0.94-- B --0.91-- C            D --0.89-- E
```

```
groups: {A, B, C}      and      {D, E}
```

The danger to watch: chains. A matches B, B matches C, but A and C are unrelated. Connected components will still put all three together, and one bad bridging record can pull hundreds of unrelated items into a single blob.

Three defences: keep the threshold high; after forming a group, verify that *every* pair inside it scores well, not just the connected ones, and split the group if not; and send any group above a size cap straight to human review.

OUT Clusters — each one a proposed single item.

IN Clusters with their scores.

DOES Sorts every cluster into one of three bands, and shows the middle band to a human.

above 0.90	auto-merge	system proceeds, fully logged
0.70 – 0.90	review queue	a person approves or rejects
below 0.70	leave alone	kept as separate items

This is not a workaround for imperfect AI. The problem statement explicitly requires a *"user validation and approval workflow"*. The human is a specified part of the design.

Who the reviewer is. A material master cataloguer at the CPSE — a domain expert who already knows SS304 is not SS316. Every company has their own; they approve only their own company's records.

What the screen shows. Everything needed to decide, in one view:

```
CPCL  BRG BALL DP GRV 6205 2RS SKF
IOCL  BEARING,BALL,6205-2RS,FAG

specs  6205 ✓   25mm ✓   52mm ✓   15mm ✓   2RS ✓
brand  SKF ⊗ FAG   (ignored)
price  same band ✓           vendor  same ✓
score  0.94

[ Approve ]   [ Reject ]   [ Skip ]
```

For a match between two different CPSEs, **both sides must approve**. No company decides on another company's data.

OUT Approved clusters move forward. Rejected ones are recorded, with the reason, and feed back into threshold tuning.

THE REVIEWER DECIDES WHETHER THIS PROJECT SUCCEEDS

The person doing the tedious work is not the person who gets the benefit — procurement gets the savings, the ministry gets the credit, the reviewer gets a queue. This is exactly how master-data projects die everywhere in the world.

So the review screen must be brutally fast: keyboard shortcuts, all evidence on one screen, no hunting for context. If a decision takes two minutes, the queue never empties and none of the accuracy matters.

IN An approved cluster of records.

DOES **Builds the golden record** by picking the best value for each field — highest confidence, or majority across the cluster.

Writes a standard description in noun-modifier format, the format industry actually uses. Once specs are typed this is just filling a template, and it is a direct, visible payoff from stage 02:

```
BEARING, BALL, DEEP GROOVE, 6205, 25X52X15 MM, 2RS SEALED
PIPE, SEAMLESS, CARBON STEEL, ASTM A106 GR B, 4 IN, SCH 40
VALVE, GATE, 6 IN, 150#, CS BODY, FLANGED
```

Classifies against UNSPSC, the international standard code hierarchy. We use it rather than inventing a taxonomy:

```
31 00 00 00  Segment      Manufacturing Components
31 17 00 00  Family       Bearings and Bushings
31 17 15 00  Class        Bearings
31 17 15 07  Commodity    Ball bearings
```

Issues the national code, with the UNSPSC class embedded so the code itself carries the category:

```
NMC - 31171500 - 000042
|           |           |
|           |           +-- serial number
|           +----- UNSPSC class (bearings)
+----- National Material Code
```

Writes the mapping table — the heart of the whole system:

national_code	cpse	original_code	status
NMC-31171500-000042	CPCL	100004521	active
NMC-31171500-000042	IOCL	MAT-88213	active
NMC-31171500-000042	ONGC	5000123	active
NMC-31171500-000042	BPCL	BRG-4471	active

OUT A national code, a golden record, and a mapping row for every source code.

NOBODY HAS TO CHANGE THEIR CODE

This is the most misunderstood part of the project. The national code does not replace anything. CPCL's storekeeper keeps typing `100004521` forever. The national code sits above the existing codes and links them, the same way Aadhaar linked existing IDs without replacing them, and UPI added a common address above bank account numbers without changing a single account.

Old codes stay. The new layer connects them. Nothing breaks.

IN National codes, mappings, and the purchase history.

DOES **Savings report.** Once records are clustered, purchase history can be grouped by national code — and this is where the project stops being a data cleanup and starts being about money:

```
NMC-31171500-000042    Ball Bearing 6205-2RS
```

```
CPCL    2,400 pcs @ Rs 1,420      Rs 34.1 L
IOCL    5,100 pcs @ Rs 1,180      Rs 60.2 L
ONGC    3,200 pcs @ Rs 1,310      Rs 41.9 L
BPCL    1,700 pcs @ Rs 1,240      Rs 21.1 L
```

```
12,400 units a year. One item. Four separate tenders.
Best price already achieved: Rs 1,180
Consolidating at that price alone: Rs 17.2 L saved
```

Rolled up across the dataset this produces the single number that should lead the pitch.

Safety interlock report. Everyone will show merges. We also show what we *refused* to merge — which is the more convincing half:

```
4,312 merges BLOCKED by safety rules
```

```
material grade mismatch (SS304 vs SS316)    1,204
pressure rating mismatch (150# vs 300#)      887
price anomaly (over 10x difference)          412
fastener grade mismatch (4.6 vs 8.8)        298
```

Audit log. Not optional — government data requires knowing who approved what and being able to undo it:

time	action	who	what	reversible
10:32:11	AUTO_MERGE	system	3 records -> NMC-...042	yes
10:45:03	MERGE_APPROVE	r.kumar	cluster #8821	yes
11:02:44	MERGE_REJECT	r.kumar	SS304 != SS316	-
11:15:20	UNMERGE	admin	reversed NMC-...042	-

Every merge also stores the signal values behind it, so the reasoning survives.

Dashboard — duplication rate, records processed, review queue depth, savings identified.

Export API — a mapping file an ERP team can load through SAP's own migration tooling, plus a clean adapter interface showing how other ERP systems plug in. Nobody expects a live SAP connection in a hackathon; they expect evidence that you know how it would work.

OUT Everything a judge sees, and everything a real CPSE would actually use.

05 The three matching signals

Why fusing three beats perfecting one

SIGNAL	WEIGHT	WHAT IT MEASURES	WHERE IT FAILS ALONE
Text	Low	Cosine similarity between description vectors	Misses SKF vs FAG entirely; wrongly merges SS304 with SS316
Specifications	Highest	Agreement between extracted attributes, with veto fields	Useless when extraction fails or the description is too sparse
Procurement	Medium	Same vendor, similar price band, similar order size, same plant	Absent for items never purchased; two different items from one vendor look alike

No single signal is reliable. Together they cover each other's blind spots, which is the whole argument for fusing them.

Why the third signal matters most for the pitch

The problem statement explicitly asks for *"historical procurement data"* to be used. Most competing teams will ignore it, because it is extra work and not obviously part of a matching engine. It does two jobs:

As evidence for a match — two items bought from the same vendor, at a similar price, in similar quantities, at the same plant, are very likely the same item.

As a guard against a wrong match — and this is the non-obvious half:

"HEX BOLT M12 X 50 SS" Rs 42

"HEX BOLT M12 X 50 SS" Rs 4,200

identical text. identical specs. 100x price gap.

these cannot be the same item.

Something real distinguishes them that the description never captured — a certified grade, an import, a special alloy. Price catches what text and specs both miss.

06

Data model

Seven tables, and what each one is for

TABLE	HOLDS	KEY COLUMNS
materials	Every raw record from every CPSE. Never modified after ingestion.	id, cpse, source code, raw_description, uom, plant
normalised	Cleaned text and canonical units, beside the raw record.	material id, clean description, canonical_uom, pack_size
specs	Extracted attributes as JSON, plus per-field confidence.	material id, category, attributes, confidence, method
embeddings	One vector per record.	material_id, vector
pair_scores	Every scored pair with its individual signal values — this is what makes a merge explainable.	a id, b id, text sim, spec sim, proc_sim, final, blocked_by
national_codes	The golden record for each unique item.	national code, unspsc, std_description, attributes
mappings	The link from every CPSE code to its national code. The core deliverable.	national code, cpse, source_code, status, approved_by
audit_log	Every action, who did it, when, and whether it can be undone.	ts, action, actor, target, evidence, reversible
purchases	Purchase history, used for signal 3 and the savings report.	cpse, source code, vendor, unit_price, qty, po_date, plant

The important property: `materials` is append-only. Every later stage writes to its own table and references back. That is what makes every merge reversible — undoing a merge means deleting mapping rows, never restoring deleted data.

07

Tech stack

Decided — do not relitigate

LAYER	CHOICE	WHY
Backend	FastAPI (Python)	Same language as the ML work; no context switching
Database	Postgres + pgvector	Business data and vectors in one place; no second system to run
Embeddings	sentence-transformers, local GPU	No API cost, no rate limits, runs on Rashmit's machine
Vector search	pgvector, HNSW index	Fast approximate nearest-neighbour. FAISS only if scale demands it later.
LLM	Claude or Gemini API	Spec extraction only, once per record at ingestion
Frontend	React	Review queue first, dashboard second
Deployment	Docker	Reproducible; matters for the finale, not for now

NONE OF THIS IS NEEDED FOR A SMALL DATASET

With a hundred records, all-pairs comparison is about five thousand operations — which pandas does instantly. No Postgres, no pgvector, no blocking, no Docker. A single Python script proves the entire logic. Build the infrastructure when the record count actually demands it, not before.

08

Who owns what

Each piece finishable on its own

PERSON	OWNS	THEIR PART IS DONE WHEN
Madhu Lead	Architecture, normaliser, spec extractor, canonicaliser	Any description goes in and correct typed specs come out, with confidence, on all six categories
Yash	Blocking, scorer, clustering	Given specs and vectors, it emits scored clusters with hard blockers enforced
Rashmit	Embeddings, evaluation harness, threshold tuning, deployment	One command prints precision, recall and duplication rate against ground truth
Aditya	Review queue UI, then the dashboard	A reviewer can clear a hundred decisions without touching the mouse
Meghna	Deck, narrative, domain research, presenting	She can answer a hostile domain question without slides
Isha	Dataset building, demo operation	200–300 real refinery items sourced, and the demo runs identically every time

REVIEW UI BEFORE DASHBOARD

A dashboard is one slide. The review queue is the actual product — it is where the only daily user of this system spends their time. If time runs short, the dashboard is what gets cut, not the review screen.

09 Build order

This sequence is deliberate — do not reorder it

#	BUILD	WHY HERE
1	Spec extractor , validated on 50 records by hand	Everything downstream consumes its output. If extraction is unreliable, nothing else can work.
2	Normaliser	Small and rule-based. Build it alongside step 1.
3	Evaluation harness	Build it early so every later change is measurable instead of guessed at.
4	Embeddings and vector storage	Needed before candidates can be shortlisted by similarity.
5	Blocking	Measure the reduction ratio here, before scoring exists to hide behind.
6	Scorer and clustering , tuned against the harness	Thresholds are chosen from measurements, never from intuition.
7	Canonicaliser — golden record, national code, UNSPSC, mappings	Only meaningful once clusters are trustworthy.
8	Review UI and dashboard	Needs real clusters to display.
9	Audit log and export API	Wraps a working system. Required for the finale, not for a first demo.

WHY THE EVALUATION HARNESS COMES THIRD

We generate our own dataset, so we know the correct answer for every record. That means we can report genuine precision and recall — most teams can only show a demo and hope.

One caveat to state honestly: we designed the corruption in that dataset, so "98% precision" really means "we reversed 98% of the errors we ourselves introduced". To harden that claim, hand-label 200–300 real records from public sources and report both numbers. The smaller, weaker real number is the more believable one — and volunteering it builds far more credibility than hiding it.

10 Rules that don't bend

Six constraints — every design decision has to survive these

1. Precision over recall, always

A missed duplicate costs an opportunity. A wrong merge can put the wrong valve in a refinery line. The two errors are not equally bad, so the

thresholds are deliberately conservative and anything uncertain goes to a human.

2. Nothing is ever destroyed

Original codes are always retained and mapped. Every merge is reversible. The raw table is append-only.

3. Every merge is explainable

Store the individual signal scores and which fields matched. If the system cannot say why two records were merged, it should not have merged them.

4. The LLM extracts; it never decides

Attributes come from the model. The merge decision is arithmetic over those attributes. This keeps results reproducible and auditable, and prevents a hallucination from becoming a merge.

5. One LLM call per record, never per pair

Extraction happens once at ingestion and the result is stored. Pairwise scoring is pure numeric maths. This is what makes the system viable at a million records.

6. No company decides for another

Within one CPSE, that company's reviewer decides alone. Across two CPSEs, both sides must approve before a national code is issued.

11 Deliberately not in scope

What we are choosing not to build, and what to say when asked

NOT BUILDING	INSTEAD
A live SAP connection	Export file plus a documented adapter interface. Judges want evidence you understand the integration, not a working ERP link.
The full UNSPSC taxonomy	The 20–30 classes our categories actually touch.
Thirty material categories	Six done properly — bearing, pipe, valve, fastener, gasket, electrical. Depth reads as competence; breadth reads as thin.
A supervised matching model	There are no training labels, so the whole system is unsupervised by necessity. Say so plainly.
Full cryptographic privacy	A practical exposure-reduction layer. Claim reduced exposure, never "cryptographically secure" — that claim will not survive a sharp question.

The roadmap item worth explaining anyway

Cross-CPSE matching without sharing raw data is the strongest answer to *"why not just buy Informatica or SAP MDG?"* Those tools need every company's data in one place, and no CPSE will hand over its prices and vendor list. That is the actual reason this has never been built, and it is architectural, not a missing feature.

The approach: each company processes its own data locally and shares only encoded values.

stays inside the company	crosses the boundary
raw description	embedding vector
exact price Rs 1,310	price band B4
vendor name SKF India	vendor hash 7b21e4
exact quantity	order-size band

Matching still works, because equal hashes prove "same vendor" without naming it, and band comparison catches a 100x price gap without revealing either price. Results are shown as a benchmark, never as attribution — a company learns that a better price exists and how far above it they are, but not who achieved it. And when fewer than three companies are in a group, the benchmark is suppressed entirely, because with only two the minimum price *is* the other party's price.

Present this as architecture with a simplified implementation. Do not overclaim it.

The project folder is currently empty — the dataset files from the planning session are not on disk, so the generator and the three CSVs have to be rebuilt first.

For a hundred-record dataset none of the infrastructure is needed. One Python script running pandas and a single loop of LLM calls demonstrates the entire logic. What matters on Tuesday is that the reasoning is visible and the numbers are real.

WHEN	WHAT	WHO
Sunday	Dataset regenerated with real refinery item names. Spec extractor running over every record. Embeddings computed. Evaluation harness reading ground truth. Deck skeleton built.	All
Monday AM	Scorer and clustering complete, hard blockers enforced, thresholds tuned against the harness.	Yash · Rashmit
Monday PM	Savings report from purchase history — roughly an hour of work and the strongest slide in the deck. Both demo cases verified by hand.	Madhu
Monday PM	Real numbers into the deck. Three full rehearsals with timing.	Meghna · Aditya

The demo sequence

Never open with two obviously similar records merging — that looks like string matching and invites the wrong question. Open with the case a conventional tool cannot solve:

1. Show the SKF and FAG records. Almost no shared words.
2. Show plain text similarity scoring 0.31 — a fuzzy matcher stops here.
3. Show the extracted specs lining up field by field, brand deliberately ignored.
4. Show the match at 0.94 confidence.
5. Then reverse it: SS304 against SS316, 0.97 text similarity, blocked on grade mismatch.
6. *"A conventional fuzzy-matching tool misses the first and wrongly merges the second. We match on specifications, not spelling."*
7. Then the numbers: records in, unique items out, duplication rate, precision, and rupees.

TWO THINGS THAT WILL OTHERWISE GO WRONG ON THE DAY

Never make a live LLM call during the demo. Save extracted specs to a JSON file beforehand and have the script read from it. Venue internet fails, APIs rate-limit, and a broken demo erases everything.

Confirm the format with the SPOC today — is a demo allowed at all, and how many minutes do you get? The whole rehearsal depends on the answer.